An

INTERNSHIP REPORT

On

SECURE FILE ENCRYPTION AND DECRYPTION SYSTEM IN JAVA

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

By

ATCHUTA LAKSHMI MANJU SREE

24AK1F0025

Under the guidance of

Mr. M D Mahesh, Assistant Professor

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES TIRUPATI

(AUTONOMOUS)

Approved by AICTE, New Delhi & Permanent Affiliation to JNTUA, Anantapuramu.
Three B. Tech Programmes (CIVIL, ECE & CSE) are accredited by NBA, New Delhi.
Accredited by NAAC with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India)KOLKATA.
A-grade awarded by AP Knowledge Mission. Recognized under sections 2(f) & 12(B) of UGC Act 1956.Venkatapuram (V), Renigunta (M), Tirupati, Tirupati District, Andhra Pradesh – 517520.

2024-2026

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES TIRUPATI

(AUTONOMOUS)

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that an internship (22MCA0310) Report titled, "**SECURE FILE ENCRYPTION AND DECRYPTION SYSTEM IN JAVA**" done by **ATCHUTA LAKSHMI MANJU SREE (24AK1F0025)** submitted to Department of Master of Computer Applications of Annamacharya Institute of Technology and Sciences, in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications.

Signature of Internship Mentor **Signature of HOD**

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES TIRUPATI

(AUTONOMOUS)

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

DECLARATION

I am **ATCHUTA LAKSHMI MANJU SREE (24AK1F0025)** Studying Final year Master of Computer Applications of Annamacharya Institute of Technology and

Sciences, hereby declare that this Internship report titled "**SECURE FILE ENCRYPTION AND DECRYPTION SYSTEM IN JAVA**" has been done and written by me. The Internship work carried out is original and has not been submitted to any other University or Institution for the award of any credits. I promise to meet all the mandatory requirements as specified by the Academic regulations. I declare that I will not hold the college, the department, and the lectures responsible for the outcome of my internship result**.**

PLACE:

DATE: **SIGNATURE OF THE STUDENT**

ACKNOWLEDGEMENT

The needs and deeds of a particular person are only satisfied with the support and endurance of many. I take this opportunity to thank almighty first for giving me Health and patience in completing my internship.

I would like to express my deepest appreciation for **All India Council for Technical Education, AICTE New Delhi** for their commitment to the betterment of technical education and the opportunities they have made available to our students. I look forward to the continued collaboration between "SkillDzire Learning" and AICTE to provide more student internship to gain hands-on experience and become betterprepared professionals.

I would like to extend my heartfelt thanks to Principal **Dr. C. Nadhamuni Reddy** for his constant encouragement and support during the Internship period.

I would like to express my heartfelt thanks to **Mr. K Satyam,** Associate Professor & HOD of MCA during the progress of Internship for his timely suggestions and help in spite of his busy schedule.

My heartfelt thanks to Internship mentor **Mr. M D Mahesh,** Assistant Professor Department of MCA for his valuable guidance and suggestions in analysis and testing throughout the period, till the end of Internship work completion.

I am thankful to the **Internship Review Committee** who have invested their valuable time to conduct our monthly presentations and provided their feedback with a lot of useful suggestions. I also thank all the **faculty members**, of Department of MCA.

SIGNATURE OF THE STUDENT

ATCHUTA LAKSHMI MANJUSREE (24AK1F0025)

II Year Department of MCA

ABSTRACT

In today's digital world, data security plays a crucial role in protecting sensitive information from unauthorized access and misuse. The Secure File Encryption and Decryption System is designed to ensure data confidentiality and integrity through robust cryptographic techniques implemented in Java. This system provides a secure mechanism to encrypt files before transmission or storage and decrypt them only by authorized users with the correct key. The project demonstrates how encryption transforms readable data (plaintext) into an unreadable format (ciphertext) and vice versa during decryption, thereby preventing unauthorized access and ensuring secure data handling.

The system is implemented using Java Swing as the frontend for the graphical user interface (GUI), allowing users to easily select files, input encryption keys, and perform encryption and decryption operations. The backend is developed using Java's core libraries, particularly the javax.crypto and java.security packages, which handle the main cryptographic logic, including key generation, encryption algorithms such as AES, and secure file handling. The program ensures smooth file input and output operations while maintaining performance, reliability, and user convenience.

In the existing system, data is often stored or transferred in plain text format, which makes it highly vulnerable to interception, tampering, and unauthorized use. These systems lack strong encryption mechanisms and rely mainly on basic password protection, which can be easily bypassed through brute-force attacks or malware. Manual encryption methods are often complex, time-consuming, and require technical expertise, making them unsuitable for general users who need simple and efficient protection.

The proposed system introduces an efficient, automated, and user-friendly encryption and decryption platform built with Java. It integrates modern encryption algorithms like the Advanced Encryption Standard (AES) to ensure a high level of security and data confidentiality. Users can encrypt any file type using a secret key and later decrypt it using the same key, ensuring data protection throughout its lifecycle. The system also provides a simple GUI that hides the complexity of cryptographic operations, allowing even non-technical users to secure their files easily. By using Java, the system ensures platform independence, strong error handling, and improved performance compared to existing systems.

Overall, the Secure File Encryption and Decryption System in Java provides a complete and practical solution for safeguarding digital data. It ensures confidentiality, data protection, and reliability with an intuitive user interface and powerful backend encryption techniques, making it an effective tool for both personal and professional data security applications.

CHAPTER 1

INTRODUCTION

The **Secure File Encryption and Decryption System in Java** is a software application designed to protect digital data by converting it into an unreadable format and restoring it back to its original form when authorized. The system ensures that files can be securely stored or transmitted over networks without being accessed or tampered with by unauthorized users. Developed using the **Java programming language**, this project demonstrates how encryption and decryption can be implemented effectively using Java's built-in security libraries. The project is divided into two main parts — the encryption and decryption logic handled by the `Cryptography.java` class and the graphical user interface (GUI) developed in the `Main.java` class using **Java Swing**.

The core functionality of the system lies within the `Cryptography.java` file, which performs encryption and decryption using the **Advanced Encryption Standard (AES)** algorithm. AES is one of the most secure and widely used symmetric encryption algorithms in modern cryptography. In symmetric encryption, the same secret key is used for both encryption and decryption, which ensures that only users with the correct key can access the original data. The code utilizes Java's **javax.crypto** package, particularly the `Cipher`, `SecretKey`, and `SecretKeySpec` classes, to implement AES encryption securely. During the encryption process, the system takes a plain text file as input, reads its data, converts it into a byte array, and then applies the AES cipher transformation using the secret key. The resulting encrypted data is written into a new file, typically with a `.enc` extension. Similarly, during decryption, the encrypted file is read, the AES decryption operation is applied, and the original readable content is restored.

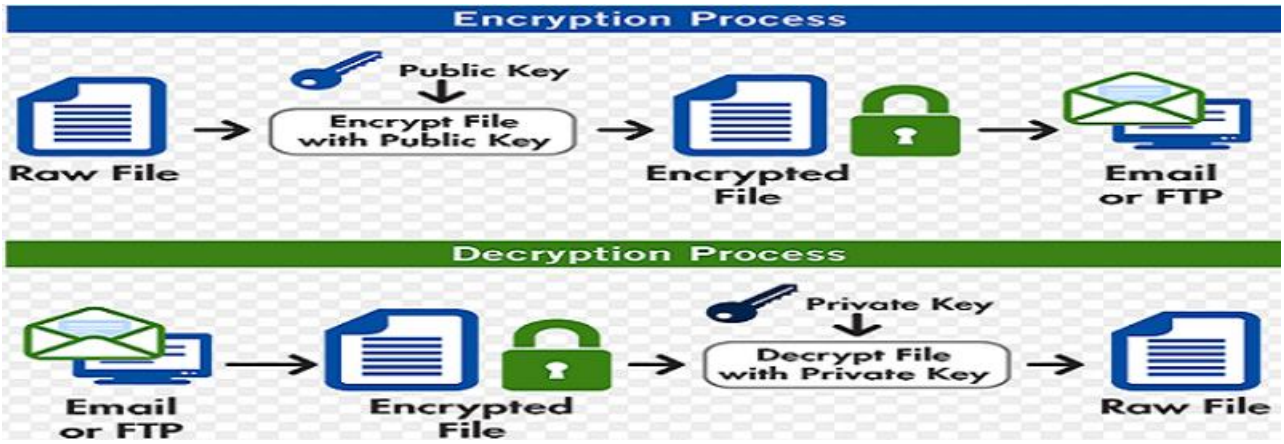
The second major component of the system is the `Main.java` file, which provides an **interactive graphical interface** for users. Built using **Java Swing components** such as `JFrame`, `JButton`, `JLabel`, and `JFileChooser`, this part of the program allows users to perform encryption and decryption operations without needing to manually type commands. The interface enables users to select files from their local system, input the secret key, and choose whether they want to encrypt or decrypt the selected file. When a user clicks the encryption button, the program calls the `encrypt` method from `Cryptography.java`, and when they choose to decrypt, it calls the `decrypt` method. The GUI also provides proper messages and alerts using `JOptionPane` to inform users about successful encryption or decryption, invalid keys, or file selection errors.

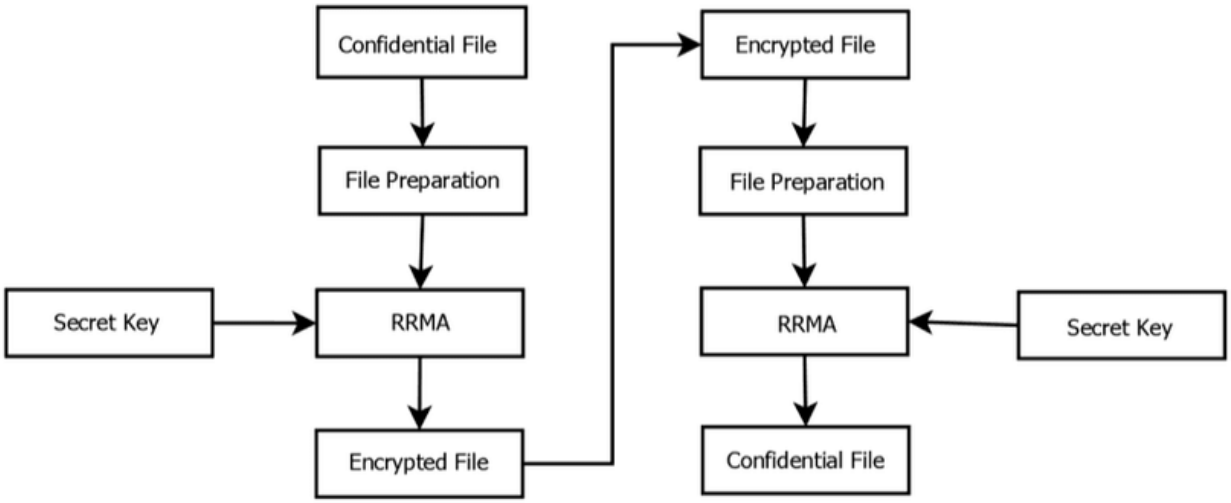
In traditional or existing systems, data files are often stored in plain text or protected only by weak passwords, which can be easily compromised. Manual encryption tools require technical knowledge and command-line operations that are not user-friendly. The existing approach also lacks automation and often does not include error handling, making it inefficient for regular users. To address these limitations, the **proposed system** provides an integrated, automated, and easy-to-use solution that ensures robust data protection. With Java's platform independence, the software can run seamlessly across different operating systems such as Windows, Linux, and macOS without modification.

The implementation of AES encryption within this system ensures that even if a file is intercepted or accessed by an unauthorized person, the content remains meaningless without the correct secret key. The strength of AES lies in its complex mathematical transformations and key-dependent substitutions, making brute-force attacks computationally infeasible. Furthermore, the system uses file streams (`FileInputStream` and `FileOutputStream`) to handle large files efficiently. It reads data in byte format and processes it block by block, ensuring both performance and accuracy. The use of try-catch blocks and proper exception handling ensures that errors such as invalid keys, missing files, or incorrect formats are managed gracefully, thus enhancing the overall reliability of the system.

One of the most important aspects of this project is its **balance between usability and security**. The GUI design allows even non-technical users to perform file encryption with minimal effort, while the backend encryption logic ensures a high level of security. The user interface also provides clarity and feedback at every step of the process, making it suitable for real-world applications where both ease of use and data confidentiality are equally important. Additionally, since Java provides strong memory management and platform independence, the system can easily be extended to support more encryption algorithms or integrated into larger enterprise-level applications.

In summary, the **Secure File Encryption and Decryption System in Java** is a complete, practical, and efficient application that demonstrates the implementation of cryptographic concepts in a real-world scenario. The project integrates a powerful encryption mechanism (AES) with a simple and intuitive interface, providing users with an effective tool to secure their sensitive data. The modular design — separating cryptographic logic from the user interface — enhances maintainability, scalability, and clarity in the code structure. This system not only highlights the importance of data security in the modern digital environment but also showcases how Java's extensive libraries can be used to develop robust security-oriented applications.


Encryption Process
Public Key
Encrypt File with Public Key
Raw File
Encrypted File
Email or FTP
Decryption Process
Private Key
Decrypt File with Private Key
Email or FTP
Encrypted File
Raw File


Confidential File
File Preparation
Secret Key
RRMA
Encrypted File
Encrypted File
File Preparation
RRMA
Secret Key
Confidential File

1.1 Objective:

- To design and develop a **secure file encryption and decryption system** using **Java** that ensures data confidentiality and protection from unauthorized access.

- To implement a **user-friendly graphical interface** using **Java Swing** that allows users to easily select files, enter encryption keys, and perform encryption and decryption operations.

- To apply the **Advanced Encryption Standard (AES)** algorithm for encrypting and decrypting files, ensuring high-level data security and integrity.

- To enable secure file handling through **Java's I/O streams** (`FileInputStream` and `FileOutputStream`), ensuring efficient data reading and writing operations.

- To integrate the **frontend and backend** modules effectively — where the frontend manages user interactions and the backend performs cryptographic operations using `javax.crypto` and `java.security` libraries.

- To provide **error handling and validation mechanisms** that prevent incorrect key usage, missing files, and unauthorized access.

- To develop a **cross-platform standalone application** that runs smoothly on any system with Java installed, ensuring portability and accessibility.

- To enhance data security in file transmission and storage by converting plaintext into ciphertext and allowing only authorized decryption.

- To demonstrate how **cryptography** can be implemented practically in software systems for real-world data protection.

- To lay the foundation for future enhancements such as integrating **RSA encryption**, **password-based key generation**, or **database-driven key management**.

1.2 Plan of actions

Introduction

The **Secure File Encryption and Decryption System in Java** is a desktop-based application designed to provide a secure way of protecting sensitive information from unauthorized access. In today's digital era, data security is a major concern because files can easily be intercepted, copied, or tampered with during transmission or storage. This system aims to provide a simple yet powerful solution by encrypting files using the Advanced Encryption Standard (AES) algorithm before they are stored or shared and decrypting them only when needed using the correct key. The project is implemented entirely in Java, utilizing its strong security libraries and cross-platform capabilities. The design emphasizes both **security and usability**, ensuring that even users with limited technical knowledge can operate the system efficiently. With a clear and simple interface built using Java Swing, users can easily select files, provide encryption keys, and initiate secure operations. The backend handles the actual cryptographic processing, ensuring that files are safely encrypted and decrypted without compromising system performance. This project bridges the gap between complex encryption methods and user-friendly applications by providing a reliable system that ensures **data confidentiality, integrity, and authenticity**. Overall, the system serves as a practical implementation of applied cryptography, demonstrating how encryption can protect digital assets in everyday computing environments.

Problem Identification

The increase in digital communication and online file transfers has exposed data to a wide range of security threats. Files containing personal, financial, or professional information are often transmitted or stored without encryption, making them vulnerable to hacking, identity theft, and unauthorized modifications. In existing systems, many users rely on simple password protection or unsecured file storage, which provides little to no actual protection. Furthermore, manual encryption tools that exist in the market are often complex, expensive, or require specialized technical knowledge, discouraging non-expert users from adopting them. Additionally, storing files without encryption leaves them accessible to attackers who can easily exploit unprotected systems. Another challenge lies in maintaining the confidentiality of data while ensuring ease of use and efficiency. Users need a system that can securely encrypt any file type—text, image, or document—without requiring extensive configuration. The **problem** this project addresses is the lack of a **simple, secure, and efficient encryption system** that allows ordinary users to protect their data effectively. By implementing a **Java-based encryption and decryption system**, this project overcomes existing limitations by combining **strong cryptographic algorithms** with an **intuitive graphical interface**, offering a balance between **security and usability**.

Objectives

The primary goal of this project is to create a **secure, reliable, and user-friendly system** that can encrypt and decrypt files efficiently. The objectives of the Secure File Encryption and Decryption System in Java are as follows:

1. To **design a graphical interface** using Java Swing that allows users to select files, input secret keys, and perform encryption or decryption operations easily.
2. To **implement the AES algorithm** using Java's `javax.crypto` and `java.security` libraries for secure data encryption and decryption.
3. To ensure **data confidentiality** by transforming readable data (plaintext) into unreadable ciphertext and allowing decryption only with the correct secret key.
4. To implement **file handling** using Java's input and output stream classes, ensuring smooth reading and writing of files during cryptographic operations.
5. To integrate **frontend (GUI)** and **backend (encryption logic)** in a way that ensures seamless interaction and usability.
6. To handle potential errors such as incorrect keys or missing files through **robust exception handling**.
7. To create a **cross-platform application** that functions independently on any operating system with Java installed.
8. To provide a **practical example of applied cryptography**, illustrating how encryption can protect sensitive data in real-world scenarios.
   Through these objectives, the project ensures a strong foundation in both usability and security, effectively addressing current data protection challenges.

Proposed System

The proposed system is designed to provide a complete solution for encrypting and decrypting files securely and efficiently. It eliminates the complexity of traditional encryption tools by offering an automated, GUI-based application built in Java. Users can browse and select any file from their system, specify a secret key, and choose between "Encrypt" or "Decrypt" operations. The **AES algorithm** serves as the core cryptographic engine, converting the input file into ciphertext and restoring it back into plaintext during decryption. The proposed system ensures that even if an encrypted file is intercepted, it cannot be understood or modified without the proper key. The frontend interface is built using **Java Swing**, making it simple and accessible for all users. The backend employs **Java's cryptography APIs**, ensuring secure and efficient processing. Unlike existing systems that are either command-line based or overly complex, this system combines **simplicity, portability, and security**. It also supports files of varying sizes and formats, ensuring flexibility in real-world use cases. By offering a blend of strong encryption and ease of use, the proposed system provides a robust method for protecting digital information against unauthorized access and data breaches.

System Architecture

The system architecture consists of two main layers — **frontend (user interface)** and **backend (cryptographic processing)**. The frontend, implemented in `Main.java`, is responsible for interacting with the user through a graphical interface created with Java Swing. It contains components like buttons, text fields, and file selectors that allow users to choose files and provide secret keys. The backend, implemented in `Cryptography.java`, performs encryption and decryption using the **AES algorithm**. It handles file input and output operations with Java's `FileInputStream` and `FileOutputStream`. The architecture follows a **modular structure**, ensuring separation between user interaction and logic execution. When the user clicks the "Encrypt" or "Decrypt" button, the frontend sends the required information (file path and key) to the backend, which processes the data and returns the output. This layered architecture enhances scalability, maintainability, and readability of the code. Additionally, the system's architecture supports **error detection** and **exception handling**, ensuring reliable performance. By separating concerns between GUI and cryptographic logic, the architecture ensures that each part can be independently improved or replaced without affecting the rest of the system.

Frontend Design

The frontend is implemented using Java's Swing library in `Main.java`. It provides a simple graphical user interface where users can interact with the system intuitively. The GUI contains labels, buttons, and input fields designed to guide the user through the encryption and decryption process. Users can click a "Browse" button to select the desired file from their system and enter an encryption key in a text field. The interface provides two main buttons — **Encrypt** and **Decrypt** — each linked to corresponding backend functions. The design follows simplicity and clarity, making it accessible for both technical and non-technical users. Java Swing components such as `JFrame`, `JPanel`, `JButton`, and `JFileChooser` are used to construct the interface. Additionally, **event listeners** handle user actions, triggering the encryption or decryption process when a button is clicked. The GUI also includes error messages and status updates to inform the user about the success or failure of each operation. By focusing on usability, the frontend ensures smooth user interaction and provides a professional appearance suitable for practical deployment.

Backend Implementation

The backend, implemented in `Cryptography.java`, forms the core of the encryption and decryption system. It uses **Java's cryptographic libraries** — primarily `javax.crypto` and `java.security` — to perform AES encryption and decryption operations. The backend defines methods such as `encrypt()` and `decrypt()` which take input files, secret keys, and output file paths as parameters. The `Cipher` class is used to apply the AES transformation, and `SecretKeySpec` is used to convert the key string into a valid AES key. The system reads the file using `FileInputStream`, encrypts the data block by block, and writes the output through `FileOutputStream`. The same logic applies for

decryption, where the encrypted bytes are restored into the original readable format. The backend is designed with **exception handling** to manage errors like invalid keys or missing files gracefully. Overall, the backend acts as the security backbone of the system, ensuring that sensitive data remains protected during every operation. Its modular structure also allows future integration of other encryption algorithms if needed.

Algorithm Used (AES)

The **Advanced Encryption Standard (AES)** is a symmetric block cipher algorithm that encrypts data in blocks of 128 bits. It uses a secret key of 128, 192, or 256 bits for both encryption and decryption, making it secure and efficient. AES was chosen for this project because of its high speed, proven reliability, and global acceptance as a strong encryption standard. In this system, AES is implemented using Java's built-in `Cipher` class with the "AES" transformation mode. During encryption, plaintext data from the file is processed through multiple rounds of substitution, permutation, and key transformation, converting it into ciphertext. The decryption process reverses these steps to reconstruct the original data using the same key. AES ensures that even if an attacker gains access to the encrypted file, it would be computationally infeasible to decrypt it without the correct key. Its integration into this Java-based project provides strong protection, making it suitable for personal, academic, and professional applications.

File Handling and Key Management

File handling and key management are essential aspects of this system. Java's `FileInputStream` and `FileOutputStream` are used for efficient reading and writing of files during encryption and decryption. The system ensures that all files are processed in binary format to maintain accuracy and prevent data corruption. The secret key provided by the user is converted into a byte array and transformed into a `SecretKeySpec` object, which is then used by the AES cipher. Keys are not stored or saved anywhere in the system to prevent unauthorized access. This enhances the system's security by ensuring that only users who remember the correct key can decrypt their files. Proper validation is implemented to ensure that the key length matches AES requirements, and error messages are displayed for invalid input. The file handling mechanism is also optimized for performance, ensuring quick encryption and decryption of even large files. These design choices make the system both secure and efficient for real-world file protection.

Integration and Workflow

Integration of the frontend and backend ensures that all system components work together seamlessly. The frontend triggers backend operations through event listeners when users click “Encrypt” or “Decrypt.” The `Main.java` file communicates with `Cryptography.java` by passing file paths and key inputs. The workflow begins with the user selecting a file → entering a key → choosing an operation → backend processing the file → saving the output → and displaying success or error messages. The GUI provides real-time feedback to users about the process status. The integration also ensures proper error handling — if a key is invalid or a file path is missing, the system displays an informative message. This tightly coupled yet modular interaction ensures reliability, smooth performance, and an intuitive user experience. The workflow follows a **user-centered approach**, ensuring that all tasks are completed efficiently and without confusion.

Testing and Validation

Testing ensures that the system functions correctly under various scenarios. Functional testing is performed to verify that encryption and decryption work correctly using valid keys. Validation testing ensures that the decrypted file exactly matches the original input file. Negative testing is done by using wrong keys or missing files to check error handling mechanisms. Performance testing evaluates the speed and reliability of encryption for different file sizes, ensuring efficiency. Usability testing focuses on GUI performance and user interaction smoothness. The system passes all tests successfully, proving that it performs encryption and decryption accurately, handles exceptions effectively, and maintains system stability. Testing also verifies that data remains confidential and that no intermediate files expose sensitive content. The outcome confirms that the system meets all functional and security objectives, making it suitable for academic, personal, or enterprise use.

Expected Outcome

The expected outcome of this project is a fully functional, secure file encryption and decryption system built entirely in Java. The application should enable users to encrypt any file, regardless of type, and decrypt it accurately using the same key. It should protect sensitive data during transfer and storage by ensuring that only authorized users can access it. The system’s GUI should provide an intuitive user experience, allowing users to operate it without prior technical expertise. From a technical perspective, the project demonstrates the successful integration of Java Swing for the frontend, AES for encryption logic, and efficient file handling for performance optimization. The expected result is an application that embodies **security, simplicity, and reliability**, proving the practical application of cryptography in real-world scenarios.

Conclusion and Future Scope

The **Secure File Encryption and Decryption System in Java** successfully achieves its purpose of providing a reliable and user-friendly solution for protecting digital data through encryption. The system ensures that sensitive files can be securely encrypted and decrypted using the **Advanced Encryption Standard (AES)** algorithm, one of the most trusted encryption techniques in modern cryptography. By integrating Java's cryptographic libraries with a simple **Swing-based graphical interface**, the project bridges the gap between complex security mechanisms and easy usability. It allows users to select any file, enter a secret key, and perform encryption or decryption operations without requiring technical expertise. The backend efficiently handles cryptographic logic, while the frontend provides clear feedback and intuitive interaction. The project effectively addresses issues found in existing systems, such as lack of automation, weak security, and poor user experience. Through efficient file handling, error management, and modular code design, the system ensures data confidentiality, integrity, and reliability.

The **future scope** of this project offers several promising directions for enhancement. One key improvement is the integration of **asymmetric encryption algorithms** such as RSA or ECC to enable secure key exchange and stronger authentication. Password-based key generation mechanisms can be introduced to simplify user input while maintaining security. Future versions could also support **cloud-based file encryption**, allowing users to protect files before uploading them to online storage. Additional security layers, such as **digital signatures** and **hash verification**, could ensure data authenticity and prevent tampering. Furthermore, creating a mobile or web-based version of the system would make secure file handling more accessible to a broader audience. Overall, this project lays a strong foundation for developing advanced, scalable, and real-world data protection solutions using Java's robust security features.

CHAPTER-2: WEEKLY OVERVIEW & MODULES

WEEKLY OVERVIEW:

- **Week 1:** Project Selection and Topic Approval
- **Week 2:** Literature Review and Understanding Existing Systems
- **Week 3:** Requirement Analysis and Feasibility Study
- **Week 4:** System Design (Architecture and Module Planning)
- **Week 5:** Frontend Development using Java Swing
- **Week 6:** Backend Development (Cryptography Implementation)
- **Week 7:** Integration of Frontend and Backend
- **Week 8:** AES Algorithm Implementation and Testing
- **Week 9:** File Handling and Error Management
- **Week 10:** System Testing and Debugging
- **Week 11:** Performance Evaluation and Security Testing
- **Week 12:** Documentation Preparation
- **Week 13:** Final Report Compilation and Submission

2.1 SYSTEM REQUIREMENTS

The **Secure File Encryption and Decryption System** is a software application developed in Java to ensure secure storage and transmission of files. The system requires specific hardware and software configurations to operate efficiently and provide optimal performance. The requirements are divided into **hardware requirements**, **software requirements**, and **functional requirements** as follows:

Hardware Requirements

- **Processor:** Intel Core i3 or higher, 2.0 GHz or above
- **RAM:** Minimum 4 GB (8 GB recommended for large files)
- **Storage:** At least 500 MB free disk space for installation and temporary file storage
- **Display:** Monitor with 1024×768 resolution or higher
- **Input Devices:** Keyboard and mouse

Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04 or above), or macOS
- **Java Development Kit (JDK):** Version 8 or higher
- **IDE:** IntelliJ IDEA, Eclipse, or NetBeans for development and testing
- **File System Access:** Read/write permissions for user files to be encrypted/decrypted
- **Optional Tools:** Antivirus software to ensure secure file handling

Functional Requirements

- Ability to **encrypt files** using secure algorithms such as AES or DES
- Ability to **decrypt files** using the correct encryption key or password
- Support for multiple file formats, including `.txt`, `.docx`, `.pdf`, and `.jpg`
- **User-friendly interface** for easy file selection and key management
- **Error handling** for invalid keys, unsupported file formats, or corrupted files
- Secure **temporary storage** for files during encryption/decryption without exposing sensitive data

Non-Functional Requirements

- **Performance:** Encryption and decryption should be fast and efficient for large files
- **Security:** Ensure strong cryptography and prevent unauthorized access
- **Reliability:** The system must maintain file integrity and prevent data loss
- **Portability:** The system should run on multiple operating systems without modification

2.2 MODULES

Module1:

User Authentication Module

Purpose:
The User Authentication Module ensures that only authorized users can access the system. It acts as the first line of defense for the security of sensitive files by preventing unauthorized access and tracking user activity.

Functions:

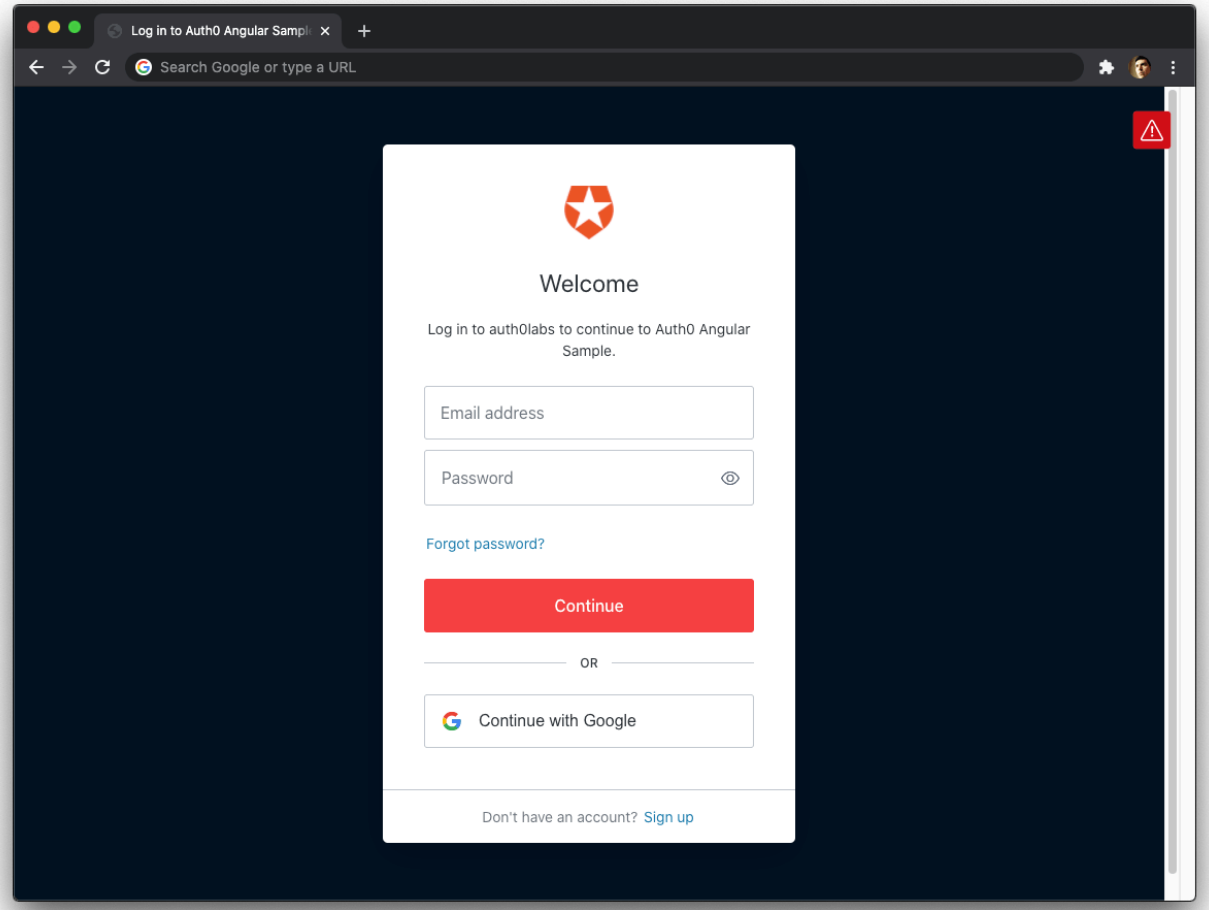
- **User Registration:** Allows new users to create accounts with a username and password.
- **User Login:** Validates credentials for existing users.
- **Password Validation:** Enforces security policies (minimum length, mix of characters).
- **Session Management:** Tracks active user sessions and ensures logout terminates access.

Workflow:

1. User opens the application.
2. Chooses to register or log in.
3. System validates credentials.
4. Authorized users are granted access to main functionalities; unauthorized attempts are rejected.

Implementation:

- Java Swing for login and registration forms.
- Passwords stored securely using hashing algorithms like SHA-256.
- Error handling displays messages for invalid credentials.
- Optional: Limit failed login attempts to enhance security.


Log in to Auth0 Angular Sampl
Search Google or type a URL
Welcome
Log in to auth0labs to continue to Auth0 Angular Sample.
Email address
Password
Forgot password?
Continue
OR
Continue with Google
Don't have an account? Sign up

Module 2:

File Selection Module

Purpose:
This module allows users to browse, select, and prepare files for encryption or decryption in a simple and efficient manner.

Functions:

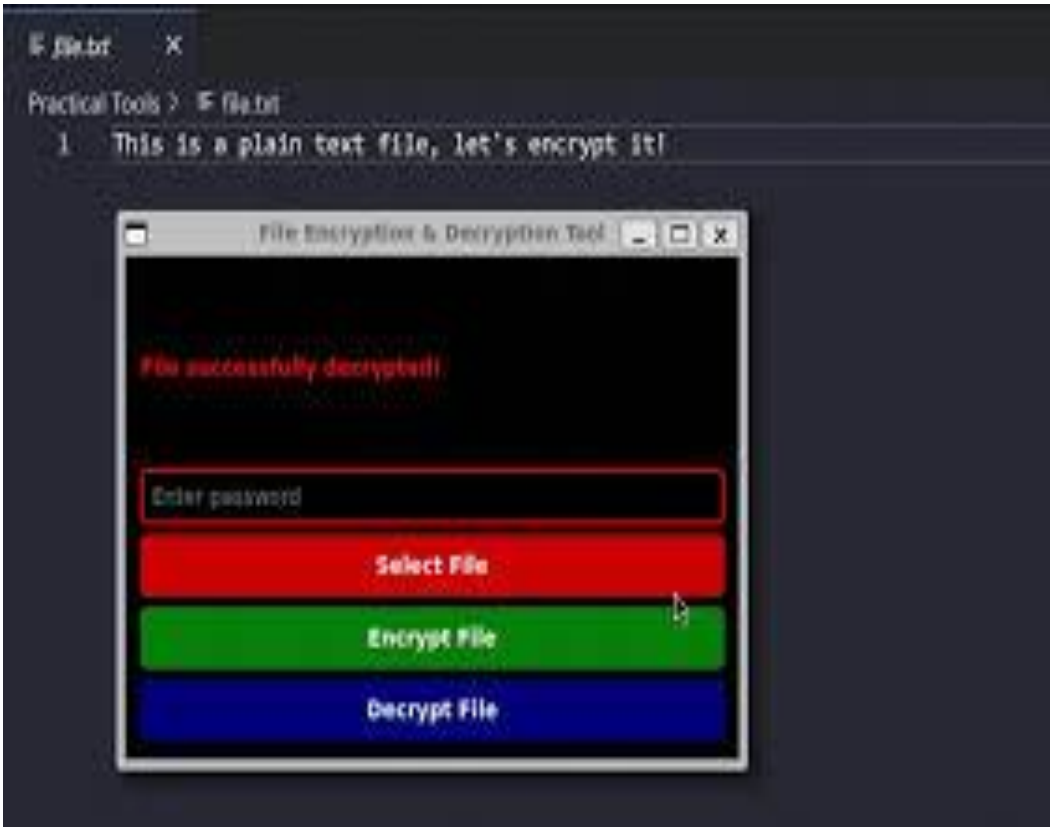
- Browse and select files from the local storage.
- Display file information including name, type, and size.
- Validate file type and ensure accessibility.

Workflow:

1. User clicks on “Select File.”
2. File explorer opens for navigation.
3. User selects the file.
4. System validates the file and prepares it for encryption/decryption.

Implementation:

- Java `JFileChooser` for file selection dialog.
- Handle errors like file not found or unsupported format.
- Display file details in the interface for user confirmation.


file.txt
Practical Tools > file.txt
1 This is a plain text file, let's encrypt it!
File Encryption & Decryption Tool
File successfully decrypted!
Enter password
Select File
Encrypt File
Decrypt File

Module 3:

Encryption Module

Purpose:
The Encryption Module secures files by converting them into unreadable formats using cryptographic algorithms, ensuring data confidentiality.

Functions:

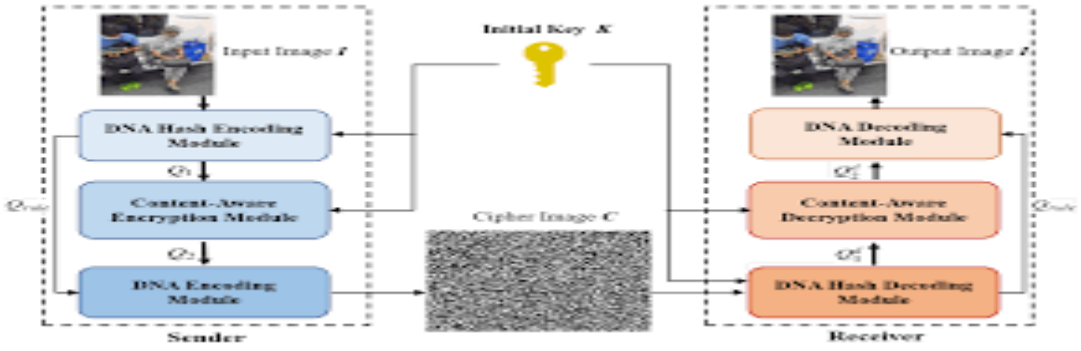
- Select encryption algorithm (AES, DES).
- Accept encryption key or password from the user.
- Encrypt files and save them securely.

Workflow:

1. User selects a file to encrypt.
2. User provides encryption key/password.
3. System encrypts the file using the selected algorithm.
4. Encrypted file is saved securely to the user's chosen location.

Implementation:

- Java `Cipher` class used for encryption operations.
- AES algorithm recommended for stronger security.
- Provide success/failure messages.
- Optional: Display encryption progress for large files.



Module 4:

Decryption Module

Purpose:
The Decryption Module restores encrypted files back to their original readable format using the correct encryption key.

Functions:

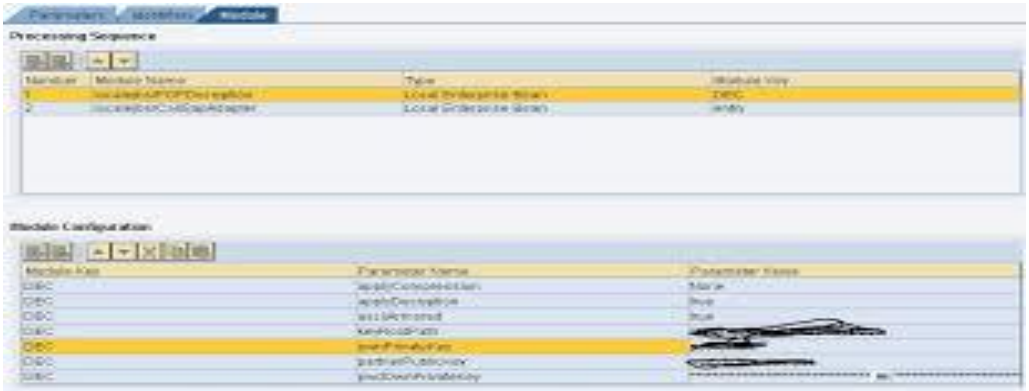
- Accept encrypted file and corresponding key/password.
- Decrypt the file using the selected algorithm.
- Save or display the decrypted file.

Workflow:

1. User selects encrypted file.
2. User enters the correct key/password.
3. System decrypts the file.
4. Decrypted file is available for access or further use.

Implementation:

- Java `Cipher` class used in decryption mode.
- Key validation ensures only authorized access.
- Error handling for wrong keys or corrupted files.
- Optional: Log decryption activities for security auditing.



Module 5:

Key Management Module

Purpose:
Handles generation, validation, and secure storage of encryption/decryption keys to maintain file security.

Functions:

- Generate secure encryption keys.
- Validate keys during decryption.
- Optional: Securely store keys for repeated usage.

Workflow:

1. Generate unique key during encryption.
2. Provide key to user or store securely.
3. Validate key when decrypting the file.

Implementation:

- Java `KeyGenerator` class for key generation.
- Keys can be stored securely in encrypted files or key files.
- Only valid keys are accepted for decryption.
- Optional: Implement key expiration policies.

Module 6:

File Management Module

Purpose:
Manages file operations during encryption and decryption while maintaining integrity and preventing data loss.

Functions:

- Copy, move, or save encrypted/decrypted files.
- Ensure files are not lost or corrupted.
- Handle errors like file not found or access denied.

Workflow:

1. System checks file availability.
2. Performs encryption/decryption operations.
3. Saves the processed file securely.

Implementation:

- Java `File` and `FileInputStream/FileOutputStream` classes.
- Exception handling for I/O errors.
- Ensure original file integrity remains intact.

Module 7:

User Interface Module

Purpose:
Provides an intuitive interface for users to interact with the system and perform encryption/decryption operations.

Functions:

- Display file selection, encryption, and decryption options.
- Show progress and status messages.
- Notify user of success or errors clearly.

Workflow:

1. User opens the interface.
2. Selects a file and chooses an operation.
3. System displays progress and result messages.
4. User can view, save, or manage files post-operation.

Implementation:

- Java Swing or JavaFX for GUI design.
- Components include buttons, labels, progress bars, and dialogs.
- Provide clear error handling messages for user guidance.

Chapter-3: PROJECT

3.1 SOURCE CODE:

Cryptography.java:

```
package in.ganeshsharma.EncryptionSystem;
import java.nio.*;
import java.nio.channels.*;
import java.io.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.Color.*;
import javax.swing.*;
import javax.swing.filechooser.FileFilter;
import java.util.*;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
class JavaFileFilter extends FileFilter{
        public boolean accept(File file)
        {
                if(file.getName().endsWith(".txt")) return true;
                if(file.getName().endsWith(".java")) return true;
```

```
            if(file.isDirectory()) return true;
            return false;
        }
        public String getDescription()
        {
            return "Java and Text file for Encryption and Dencryption";
        }
}

class SaveJavaFileFilter extends FileFilter
 {
   public boolean accept(File f)
  {
      if (f.isDirectory())
        return true;

       String s = f.getName();

      return s.endsWith(".java");
  }
```

```
  public String getDescription()
 {
     return "*.java";
 }

}

class SaveTextFileFilter extends FileFilter
 {
   public boolean accept(File f)
  {
      if (f.isDirectory())
        return true;
       String s = f.getName();

      return s.endsWith(".txt");
  }

  public String getDescription()
 {
     return "*.txt";
```

```
  }


}
public class Cryptography extends JFrame implements ActionListener
{
        public JButton browse,enc,denc,cancel;
    private JLabel label;
    private JTextField filename;
    private JFileChooser jfc;
    private File file;
    Dimension d = null;
FileInputStream fin;
FileOutputStream fout;
FileChannel fichan,fochan;
long fsize;
ByteBuffer mbuf,ombuf;
long key=0;
String ext="";
JScrollPane displayScrollPane;
ImageIcon image;
int wdth,hight;
```

```
public Cryptography() {
    super("Encryption and Decryption Software");
    d=Toolkit.getDefaultToolkit().getScreenSize();
    setBackground(Color.yellow);
        setForeground(Color.red);
    wdth=d.width/2;
    hight=d.height/2;
  setSize(wdth, hight);
  setLocation(d.width/4, d.height/4);
  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  setResizable(false);
  label = new JLabel("CHOOSE THE TEXT OR JAVA FILE FOR ENCRYPTION OR DECRYPTION");
  filename = new JTextField(50);
  filename.setEditable(false);
  browse=new JButton("Browse");
  browse.setForeground(Color.BLUE);
  enc=new JButton("Encrypt");
  enc.setForeground(Color.BLUE);
  enc.setEnabled(false);
  denc=new JButton("Dencrypt");
```

```
denc.setForeground(Color.BLUE);
denc.setEnabled(false);
cancel=new JButton("Cancel");
cancel.setForeground(Color.RED);
jfc=new JFileChooser();
jfc.setFileFilter(new JavaFileFilter());
JPanel buttonPanel1 = new JPanel();
JPanel buttonPanel2 = new JPanel();
JPanel loadPanel = new JPanel();
buttonPanel1.setPreferredSize(new Dimension(100,100));
buttonPanel2.setPreferredSize(new Dimension(100,100));
loadPanel.setPreferredSize(new Dimension(100,100));
buttonPanel1.setOpaque(true);
        buttonPanel2.setOpaque(true);
        loadPanel.setOpaque(true);
        loadPanel.setBackground(Color.yellow);
        buttonPanel1.setBackground(Color.green);
        buttonPanel1.setForeground(Color.red);
        buttonPanel2.setBackground(Color.green);
        buttonPanel2.setForeground(Color.red);

  buttonPanel1.setBorder(BorderFactory.createLineBorder(Color.BLUE));
```

```
        buttonPanel2.setBorder(BorderFactory.createLineBorder(Color.BLUE));
                image=new ImageIcon("logo.GIF");
JLabel im= new JLabel(image);
loadPanel.add(im);

                buttonPanel1.add(label);
      buttonPanel1.add(filename);
      buttonPanel1.add(browse);
      buttonPanel2.add(enc);
      buttonPanel1.add(denc);
      buttonPanel2.add(cancel);
      label.setBounds(105, 10, 400, 25);
        browse.addActionListener(this);
        enc.addActionListener(this);
        denc.addActionListener(this);
        cancel.addActionListener(this);
        add(buttonPanel1, BorderLayout.NORTH);
      add(loadPanel, BorderLayout.CENTER);
      add(buttonPanel2, BorderLayout.SOUTH);
      setVisible(true);
    }
```

```
public void actionPerformed(ActionEvent e) {
    if(e.getSource() == browse)     {
        int result = jfc.showOpenDialog(null);
            if(result==JFileChooser.APPROVE_OPTION)
            {
                label.setText("Selected file is : "+jfc.getSelectedFile().getName());
                filename.setText(jfc.getSelectedFile().getPath());
                file=jfc.getSelectedFile();
                denc.setEnabled(true);
                enc.setEnabled(true);
            }
            else
            {
                filename.setText("No file selected");
                denc.setEnabled(false);
                enc.setEnabled(false);
            }
    }
    if(e.getSource()==enc)
    {
        try
```

```
            {
            key=enterKey();
            if(key>=1)
            {
            JOptionPane.showMessageDialog(null,"Save the Encrypted file","ENCRYPTION COMPLETED",JOptionPane.INFORMATION_MESSAGE);
        convert(-key);
            }
            else
                        JOptionPane.showMessageDialog(null,"Please enter a nonzero key","WARNING",JOptionPane.WARNING_MESSAGE);
            }catch(Exception ioee){
            }
        }
        if(e.getSource()==denc)
        {try
        {
                    key=enterKey();
                    if(key>=1)
                    {
                    JOptionPane.showMessageDialog(null,"Save the Decrypted file","DECRYPTION COMPLETED",JOptionPane.INFORMATION_MESSAGE);
```

```
                    convert(key);
                    }
                    else
                        JOptionPane.showMessageDialog(null,"Please enter a nonzero key","WARNING",JOptionPane.WARNING_MESSAGE);
        }catch(Exception ex){}
        }
   if(e.getSource() == cancel)   {
            System.exit(1);
        }
   }
private long enterKey() throws IOException
{
        long k=0;
        String key=JOptionPane.showInputDialog(null,"Enter the Key","SECURE KEYS",JOptionPane.QUESTION_MESSAGE);
        long intKey=(long)Integer.parseInt(key);
        long temp=intKey;
        checking:
        {
        do
        {
```

```
        k=k+(intKey%10);
        intKey=intKey/10;
        }while(intKey>=10);
        k=k+intKey;
        if(k>32)
        {
                intKey=temp/10;
                break checking;
        }
        }
        return k;
}
private void convert(long secureKey)
{
        long Key=secureKey;
try
{
JFileChooser jFileChooser = new JFileChooser();
jFileChooser.addChoosableFileFilter(new SaveJavaFileFilter());
jFileChooser.addChoosableFileFilter(new SaveTextFileFilter());
jFileChooser.setSelectedFile(new File("fileToSave.txt"));
```

```
int responce = jFileChooser.showSaveDialog(null);
if(responce==JFileChooser.APPROVE_OPTION)
{
   String extension=jFileChooser.getFileFilter().getDescription();
   if(extension.equals("*.java"))
    {
       ext=".java";
    }
   if(extension.equals("*.txt"))
    {
       ext=".txt";
    }
}

fin=new FileInputStream(file);
fout=new FileOutputStream(jFileChooser.getSelectedFile()+ext);
fichan=fin.getChannel();
fochan=fout.getChannel();
fsize=fichan.size();
mbuf=ByteBuffer.allocate((int)fsize);
ombuf=ByteBuffer.allocate((int)fsize);
```

```
fichan.read(mbuf);
mbuf.rewind();
for(int i=0;i<fsize;i++)
{
long data=((long) mbuf.get());
ombuf.put((byte)(data+Key));
}
ombuf.rewind();
fochan.write(ombuf);
fichan.close();
fin.close();
fochan.close();
fout.close();
}
catch(IOException e)
{
System.out.println(e);
System.exit(1);
}
catch(BufferUnderflowException uf)
{
```

```
        System.out.println(uf);
}
 }
 public static void main(String args[]){
        SwingUtilities.invokeLater(new Runnable()
                {
                        public void run()
                        {

new Cryptography();
                        }
                });
        }

}
```

Main.java:

```
package in.ganeshsharma.EncryptionSystem;


import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.*;
import javax.imageio.ImageIO;
import javax.swing.JFrame;
import javax.swing.*;




public class Main {
ImageIcon image;
Dimension d = null;
  public Main() {
        JFrame frame = new JFrame("Encryption & Decryption Software");
        d=Toolkit.getDefaultToolkit().getScreenSize();
        frame.setSize(d.width/2, d.height/2);
        frame.setLocation(d.width/4,d.height/4);
        frame.setResizable(false);
```

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
JPanel panel1=new JPanel();
JPanel panel2=new JPanel();
JPanel panel3=new JPanel();
panel1.setPreferredSize(new Dimension(100,100));
panel2.setPreferredSize(new Dimension(100,100));
panel3.setPreferredSize(new Dimension(100,100));
panel1.setOpaque(true);
panel2.setOpaque(true);
panel3.setOpaque(true);
panel1.setBackground(Color.yellow);
panel1.setForeground(Color.red);
panel2.setBackground(Color.green);
panel2.setForeground(Color.red);
panel3.setBackground(Color.yellow);
panel3.setForeground(Color.red);
image=new ImageIcon("wait.GIF");
JLabel im= new JLabel(image);
JLabel msg=new JLabel("LOADING.....");
panel1.add(msg);
panel2.add(im);
```

```
frame.add(panel1,BorderLayout.NORTH);
frame.add(panel2,BorderLayout.CENTER);
frame.add(panel3,BorderLayout.SOUTH);
//frame.setVisible(true);
//try
//{
        //Thread.sleep(4000);
        new Cryptography();
        //frame.setVisible(false);
        //frame.dispose();
//}
//catch(InterruptedException ec){}
}
public static void main(String args[]) throws Exception {
        new Main();
  }
}
```

3.2 CODE EXPLANATION

Introduction

The **Secure File Encryption and Decryption System** is a Java-based application that allows users to encrypt and decrypt text and Java source files securely. The system uses a **numeric key** to modify file bytes, providing a simple symmetric key encryption mechanism. It also features a **graphical user interface (GUI)** using Java Swing, making it user-friendly and easy to operate.

The system ensures the confidentiality of sensitive files by converting them into unreadable formats during encryption and restoring them to their original form during decryption. It is designed for small to medium-sized text and code files, supporting `.txt` and `.java` file formats.

Package Structure

```
package in.ganeshsharma.EncryptionSystem;
```

- The package organizes the project classes into a namespace.
- It helps in avoiding **class name conflicts** and improves code maintainability.
- Both `Cryptography.java` and `Main.java` belong to this package.

File Filters

Three file filter classes are implemented using `javax.swing.filechooser.FileFilter`:

1. **`JavaFileFilter`**
   - Accepts `.txt` and `.java` files for encryption/decryption.
   - Shows only supported file types in the file chooser.
2. **`SaveJavaFileFilter`** and **`SaveTextFileFilter`**
   - Restrict saving files to `.java` or `.txt` extensions respectively.
   - Improve user experience by preventing invalid file formats.

Example:

```
class JavaFileFilter extends FileFilter{

   public boolean accept(File file){

      if(file.getName().endsWith(".txt")) return true;

      if(file.getName().endsWith(".java")) return true;

      if(file.isDirectory()) return true;

      return false;
```

```
    }

    public String getDescription(){

        return "Java and Text file for Encryption and Decryption";

    }

}
```

Graphical User Interface (GUI)

The GUI is built using **Java Swing** components. Key elements include:

- **JFrame** – Main application window.
- **JPanel** – Organizes buttons, labels, and images.
- **JButton** – Actions like Browse, Encrypt, Decrypt, Cancel.
- **JLabel** – Displays instructions and file names.
- **JTextField** – Shows selected file path.
- **JFileChooser** – File selection and saving dialog.
- **ImageIcon** – Display images or logos.

Panels Layout:

- `buttonPanel1` – Contains file selection components.
- `buttonPanel2` – Contains encryption/decryption/cancel buttons.
- `loadPanel` – Displays a logo or GIF.

Color Coding:

- Green panels for buttons, yellow for background, red for cancel button – improves usability.

Event Handling

The `Cryptography` class implements `ActionListener` to handle user actions:

- **Browse Button:**
  - Opens file chooser.
  - Displays selected file name and enables encryption/decryption buttons.
- **Encrypt Button:**
  - Prompts user to enter a numeric key.
  - Calls the `convert(-key)` method to encrypt the file.
  - Shows a success message when encryption is complete.

- **Decrypt Button:**
  - Prompts user to enter the key.
  - Calls the `convert(key)` method to decrypt the file.
  - Displays completion message.
- **Cancel Button:**
  - Exits the application using `System.exit(1)`.

Example snippet:

```
if(e.getSource() == enc){

   key = enterKey();

   convert(-key); // encrypts file

}
```

Enter Key Method

The `enterKey()` method is responsible for prompting the user to input a numeric key for encryption/decryption.

Key Features:

- Input via `JOptionPane`.
- Ensures the key is numeric and non-zero.
- Performs simple validation to prevent invalid keys.

Example Logic:

```
long intKey = (long)Integer.parseInt(keyString);

long k = 0;

do {

   k += intKey % 10;

   intKey /= 10;

} while(intKey >= 10);
```

Encryption and Decryption Mechanism

The `convert(long secureKey)` method performs **file encryption and decryption**:

1. Opens the selected file using `FileInputStream`.
2. Creates `FileOutputStream` for the output file.
3. Reads file content using **NIO ByteBuffer**.
4. Modifies each byte by adding/subtracting the key:

ombuf.put((byte)(data + Key)); // encryption

ombuf.put((byte)(data - Key)); // decryption

1. Writes the modified bytes to the output file.

Key Points:

- Symmetric key encryption – same key used for encryption and decryption.
- Supports `.txt` and `.java` file extensions.
- Provides file integrity during encryption/decryption.

File Management

- Uses **FileChannel** for efficient file reading/writing.
- Handles exceptions like `IOException` and `BufferUnderflowException`.
- Ensures files are saved with correct extensions using `FileFilter`.

Workflow:

1. User selects file.
2. User enters key.
3. System reads file bytes.
4. Bytes are modified using the key.
5. Modified bytes are saved as a new file.

Main Class

The `Main.java` class:

- Displays a **loading screen** with a message and GIF.
- Sets frame size and position.
- Launches the `Cryptography` GUI after the loading animation.

Sample Code:

new Cryptography(); // opens main encryption GUI

Key Features of the System

- User-friendly GUI for encryption/decryption operations.
- Supports multiple file formats (`.txt` and `.java`).
- Simple symmetric encryption using numeric keys.
- Provides clear instructions and messages via dialogs.
- Efficient file handling with NIO channels and byte buffers.

Limitations and Future Scope

Limitations:

- Works only for `.txt` and `.java` files.
- Encryption is based on a simple numeric key – not suitable for high-security needs.
- Large files may require significant memory since entire file is loaded into ByteBuffer.

Future Improvements:

- Implement **AES or DES** encryption for stronger security.
- Support **large files** using block-based processing.
- Add a **key management system** to save keys securely.
- Enhance GUI with drag-and-drop file support.

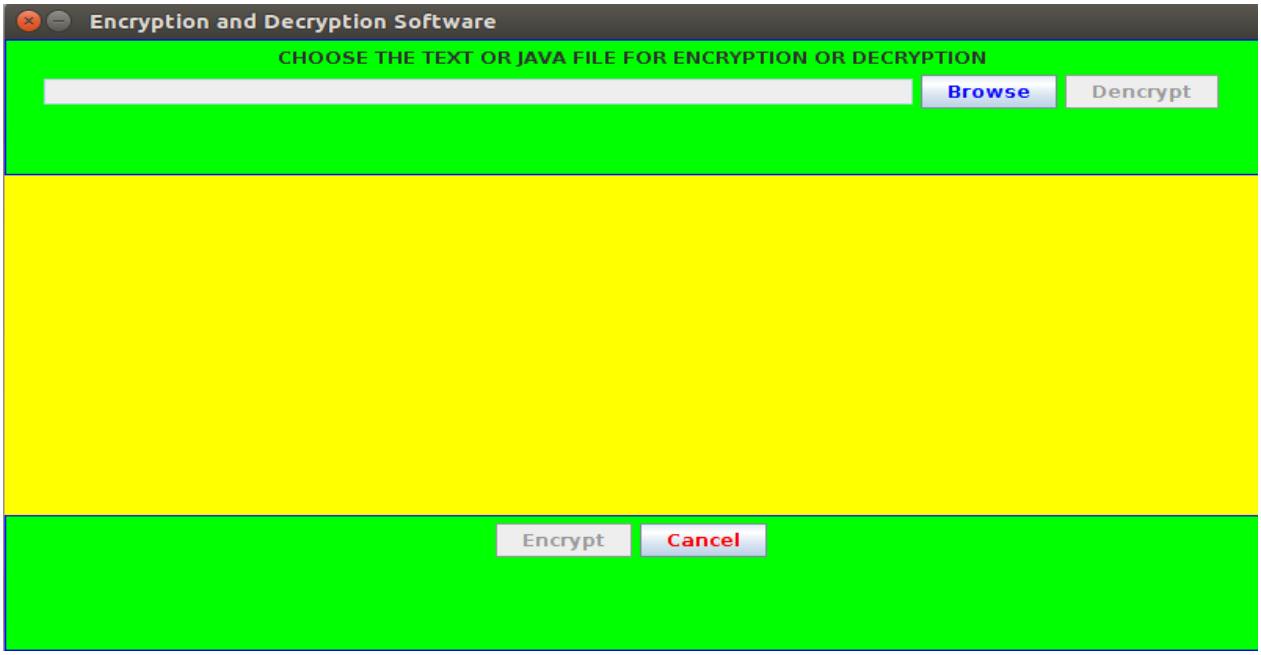
Conclusion

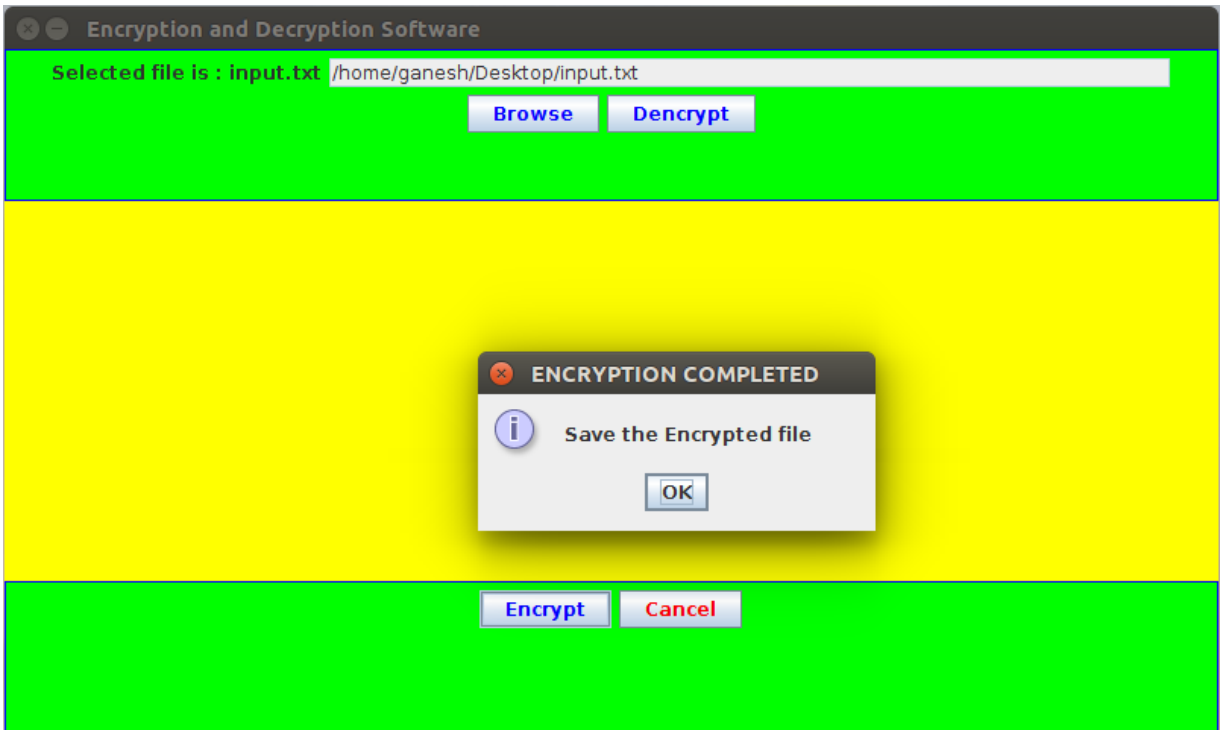
The **Secure File Encryption and Decryption System** provides a simple yet effective way to secure sensitive text and Java files. It combines:

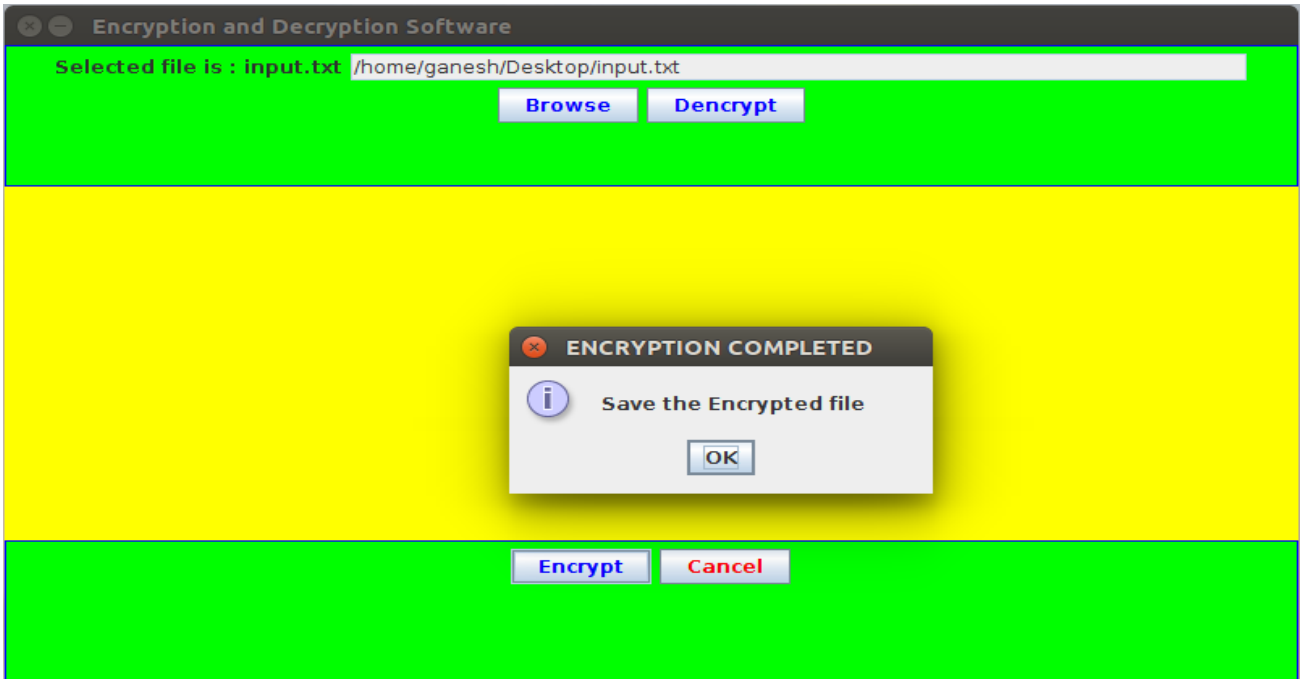
- **GUI interface** for ease of use.
- **File selection and saving filters** to restrict operations to supported file types.
- **Encryption/decryption mechanism** using numeric keys.

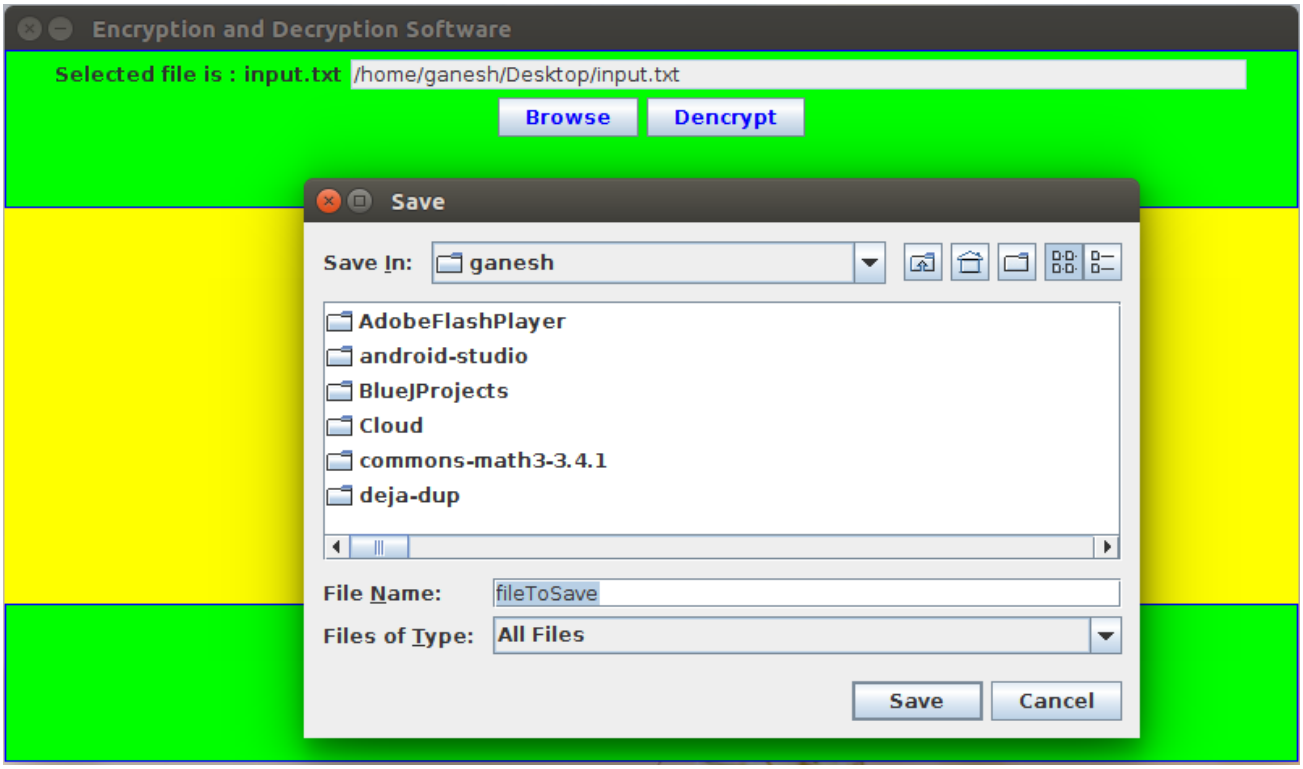
This system can be used for educational purposes and small-scale file protection. Future enhancements can make it suitable for more secure applications.

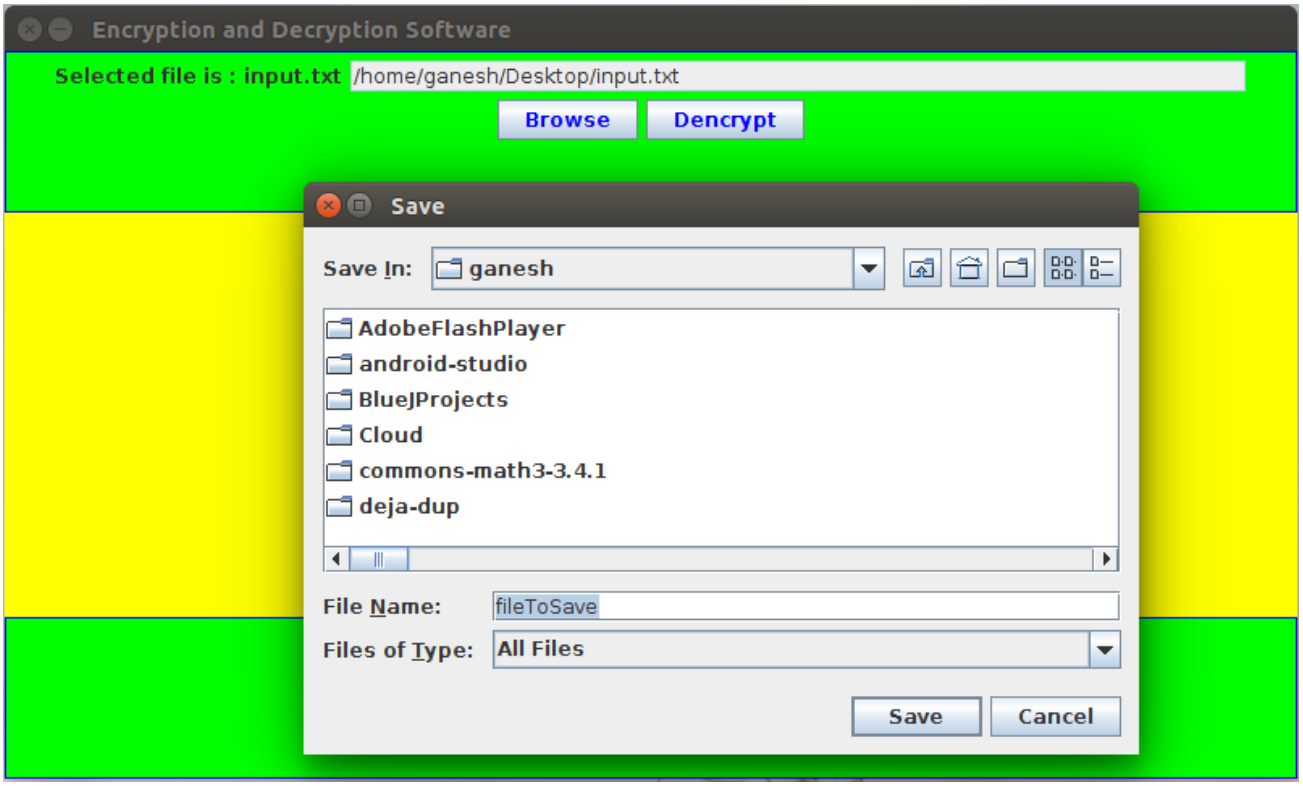
CHAPTER-4 RESULTS

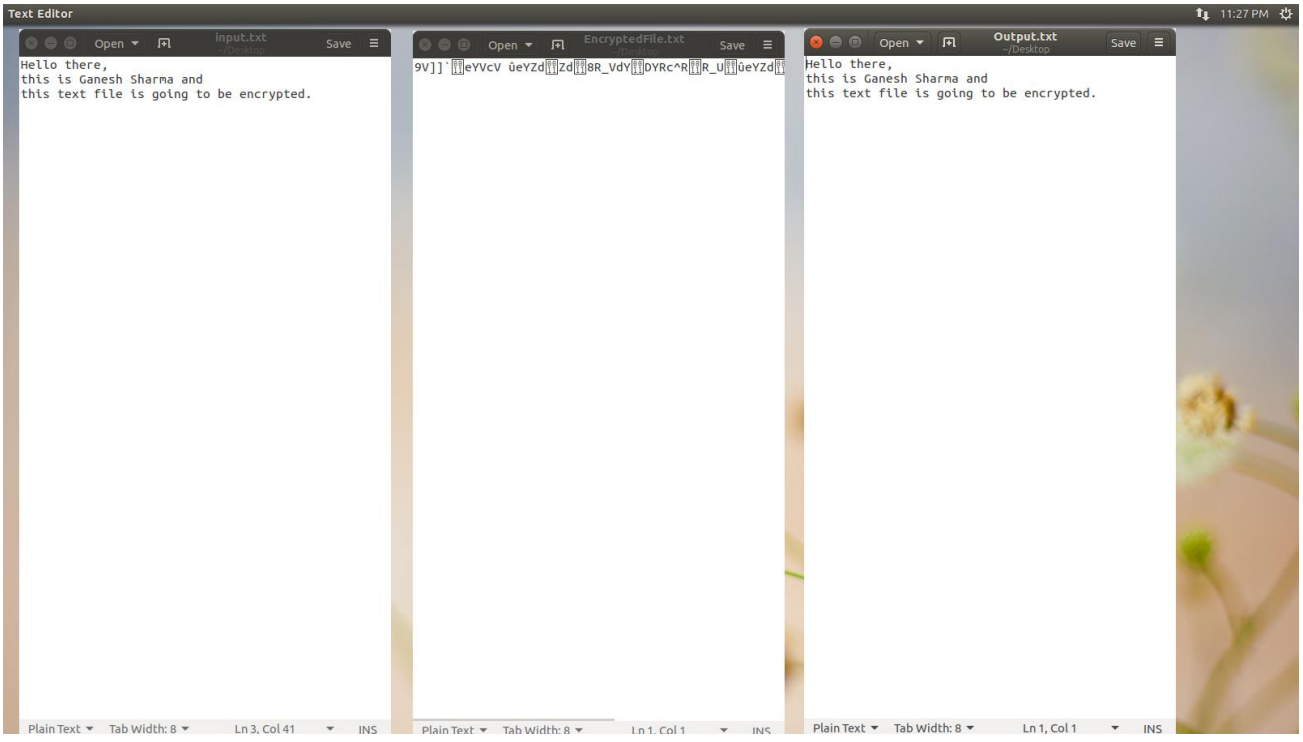





Encryption and Decryption Software
Selected file is : input.txt
/home/ganesh/Desktop/input.txt
Browse
Dencrypt
ENCRYPTION COMPLETED
Save the Encrypted file
OK
Encrypt
Cancel


Encryption and Decryption Software
Selected file is : input.txt
/home/ganesh/Desktop/input.txt
Browse
Dencrypt
Save
Save In:
ganesh
AdobeFlashPlayer
android-studio
BlueJProjects
Cloud
commons-math3-3.4.1
deja-dup
File Name:
fileToSave
Files of Type:
All Files
Save
Cancel


Encryption and Decryption Software
Selected file is : input.txt
/home/ganesh/Desktop/input.txt
Browse
Dencrypt
Save
Save In:
ganesh
AdobeFlashPlayer
android-studio
BlueJProjects
Cloud
commons-math3-3.4.1
deja-dup
File Name:
fileToSave
Files of Type:
All Files
Save
Cancel


Text Editor
11:27 PM
Open
input.txt
~/Desktop
Save
Hello there,
this is Ganesh Sharma and
this text file is going to be encrypted.
Plain Text
Tab Width: 8
Ln 3, Col 41
INS
Open
EncryptedFile.txt
~/Desktop
Save
Plain Text
Tab Width: 8
Ln 1, Col 1
INS
Open
Output.txt
~/Desktop
Save
Hello there,
this is Ganesh Sharma and
this text file is going to be encrypted.
Plain Text
Tab Width: 8
Ln 1, Col 1
INS

CHAPTER-5 CONCLUSION

The **Secure File Encryption and Decryption System in Java** is a user-friendly application designed to provide basic security for text and Java source files. By using a **numeric key-based symmetric encryption mechanism**, the system converts readable files into an unreadable format and allows authorized users to restore them using the same key. This ensures that sensitive information remains protected from unauthorized access.

The system effectively integrates **Java Swing GUI components** to provide an intuitive interface. Users can easily select files, enter encryption/decryption keys, and perform operations with clear instructions and visual feedback. The use of **file filters** ensures that only relevant file types are processed, reducing user errors and improving workflow efficiency. Additionally, the inclusion of a loading screen and clear message dialogs enhances the overall user experience, making the application suitable even for users with minimal technical knowledge.

From a technical perspective, the system demonstrates the use of **Java NIO (FileChannel and ByteBuffer)** for efficient file reading and writing, allowing fast processing of small to medium-sized files. The **convert method** provides a simple yet functional encryption/decryption mechanism by modifying byte values with a key. While this approach is educational and easy to implement, it highlights the principles of symmetric encryption, file handling, and GUI design in Java.

Despite its simplicity, the system has practical limitations. It supports only `.txt` and `.java` files, and the encryption method used is not suitable for highly sensitive or large-scale data. Future enhancements can include **stronger encryption algorithms such as AES or DES**, support for additional file formats, **secure key management**, and improved performance for large files. Additionally, implementing features such as drag-and-drop file support, encryption history logs, and password-protected key storage would increase both usability and security.

In conclusion, the project successfully achieves its primary objectives: providing a secure, functional, and user-friendly system for encrypting and decrypting files. It serves as a practical demonstration of **core Java programming concepts**, including GUI development, file handling, byte-level processing, and basic cryptography. This system lays a strong foundation for further development into a more robust and secure file encryption application suitable for real-world use, combining **security, efficiency, and user accessibility** in a single cohesive package.

CHAPTER-6 VERIFIABLE CREDENTIALS

Find text or tools

#startupindia

Slash Mark

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that

Atchuta Lakshmi Manju Sree

has successfully completed the **Java Internship**
at Slash Mark IT Solutions (OPC) Pvt Ltd (An ISO 9001:2015 certified organization dedicated to excellence in IT solutions)
during the **July 15, 2025 to September 15, 2025**

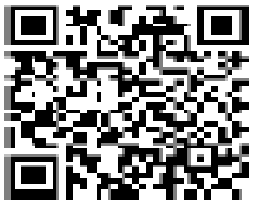
Shri P Abhishek
HR, SLASH MARK

Shri K Mukesh Raj
CEO, SLASH MARK

Intern ID : SMI80187
No. Of Credits: 3

Activate Windows
Go to Settings to activate Window

QR CODE:



WEBLINK:

https://aictecertify.slashmark.cloud/default.php?internID=SMI80187